

Manual Técnico – Anisotropia Notacional



Versão: 2.1

Data: Abril 2026

Âmbito: Fórmulas, algoritmos e convenções **exactamente** como implementados em `anisotropia/` (Python 3.10+), mais tutorial operacional. **Este é o único manual técnico do projecto.**

Ficheiros de referência: `anisotropia/metrics.py`, `parsing.py`, `windowing.py`, `visualizations.py`, `report.py`, `Anisotropia.py`.

MathJax em sites tipo Stack Exchange: blocos de equação usam `$$` em linha própria (não o delimitador LaTeX com parêntesis recto de abertura: o carácter `[` é interpretado como início de hiperligação Markdown). *Inline:* preferir `$ ... $` em cópias para esses sites.

Índice

1. [Resumo e notação](#)
2. [Pipeline de dados](#)
3. [Eventos, transições e pesos](#)
4. [Padronização e tensor de estrutura](#)
5. [Métricas escalares](#)
6. [Intervalos de confiança \(bootstrap\)](#)
7. [Agregações 2A e 2B](#)
8. [Conflito direccional](#)
9. [Janelamento temporal](#)
10. [Visualizações \(geometria\)](#)
11. [Algoritmos \(pseudocódigo\)](#)
12. [Tutorial](#)
13. [Apêndices](#)
14. [Bibliografia](#)

1. Resumo e notação

1.1 Objectivo

O sistema mede **anisotropia notacional**: até que ponto as transições melódicas ($\Delta t, \Delta p$) entre notas consecutivas concentram uma **direcção preferencial** no plano tempo–altura, usando um **tensor de estrutura** 2×2 (analogia com tensor de estrutura em visão computacional) e **estatística circular** para coerência angular.

1.2 Tabela de símbolos

Símbolo	Significado
(i)	Índice de transição dentro de uma janela
(j)	Índice de instrumento / parte
(w)	Índice de janela (segmento temporal)
(p)	Altura em semitons (MIDI float)
ql	Offset em quarter lengths (music21)
(t)	Tempo em segundos (quando secondsMap existe)
Δp_i	$(p_{i+1} - p_i)$ na cadeia de eventos colapsados
Δt_i^{ql}	$\mathrm{ql}_{i+1} - \mathrm{ql}_i$
$\Delta t_i^{\mathrm{sec}}$	$(t_{i+1} - t_i)$
(w _i)	Peso não-negativo da transição (i)
(ϵ)	(10^{-9}) — constante EPSILON no código
$\mathbf{J}$	Matriz 2×2 simétrica (tensor de estrutura ponderado)
$\lambda_1 \geq \lambda_2$	Autovalores de $\mathbf{J}$

1.3 Distinção crítica (implementação)

- **(D)** e **(τ)** usam apenas Δp_i e pesos (w_i) (não usam o tensor).
- **($\mathbf{J}$)**, **($\mathbf{A}_{\mathrm{tensor}}$)** e **(μ)** usam o par $((v_{1,i}, v_{2,i}))$ onde, se standardize=True, (v₁, v₂) são **(Δt)** e **(Δp)** padronizados (ver §4).
- **(R)** e os ângulos (θ_i) usam **sempre** os valores **originais** (não padronizados):

$$\theta_i = \operatorname{atan2}(\Delta p_i, \Delta t_i^{\star})$$

com $\Delta t_i^{\star} = \Delta t_i^{\mathrm{ql}}$ se time_axis="ql" e $\Delta t_i^{\mathrm{sec}}$ se time_axis="sec".

- O modo `w_min` calcula o peso a partir de **duração em ql** e (Δt^{ql}) apenas (não usa segundos); ver §3.3.

2. Pipeline de dados

MusicXML (bytes)

- `parse_musicxml` → `Dict[parte, List[Event]]`
- ordenar/colapsar simultâneos por `ql`
- `transitions_from_events` → `DataFrame` por parte
- `window_slices_for_part` (parte referência) → etiquetas de janela
- por janela: filtrar transições, `compute_metrics_from_transitions`
- opcional: `aggregate_2A`, `aggregate_2B`, `compute_directional_conflict`
- relatório (`generate_report`), figuras (`visualizations`)

A **parte referência** para construir a lista de janelas é a parte com **mais transições**; as outras partes são cortadas às **mesmas** janelas (por medida, tempo ou índice de transição conforme o modo).

3. Eventos, transições e pesos

3.1 Evento

Após parsing, cada evento tem campos $(t, \mathrm{ql}, \mathrm{dur_ql}, p, \mathrm{meas})$. Acordes no mesmo instante (ql) são colapsados:

$$p_{\mathrm{col}} = \frac{1}{|\mathcal{S}|} \sum_{e \in \mathcal{S}} p_e, \quad \mathrm{dur}_{\mathrm{col}} = \max_{e \in \mathcal{S}} \mathrm{dur_ql}(e)$$

com $(\mathcal{S})$ o conjunto de eventos no mesmo (ql) (tolerância (ε)).

3.2 Transição

Para eventos consecutivos (a $\rightarrow$ b) na sequência ordenada por (ql, p) :

$$\Delta p = p_b - p_a, \quad \Delta t^{\mathrm{ql}} = \mathrm{ql}_b - \mathrm{ql}_a, \quad \Delta t^{\mathrm{sec}} = t_b - t_a$$

$$w_i^{\mathrm{dur}} = \max(\mathrm{dur_ql}(a), 0)$$

$$w_i^{\mathrm{min}} = \max\left(\min(\mathrm{dur_ql}(a), \Delta t^{\mathrm{ql}} \text{ se } \Delta t^{\mathrm{ql}} > 0 \text{ senão } \mathrm{dur_ql}(a)), 0\right)$$

Nota: (w^{min}) está definido em **quarter lengths**, independentemente de o eixo temporal da métrica ser `ql` ou `sec`.

3.3 Selecção de colunas

- `weight_mode="dur"` → coluna `w_dur`
- `weight_mode="min"` → coluna `w_min`
- `time_axis="ql"` → ($\Delta t_i = \Delta t_i^{\mathrm{ql}}$) nas fórmulas do tensor e de (θ)
- `time_axis="sec"` → ($\Delta t_i = \Delta t_i^{\mathrm{sec}}$)

3.4 Filtragem (`compute_metrics_from_transitions`)

1. Remover linhas com `dp` ou (Δt) não finitos.
2. Remover linhas com ($\Delta t \leq 0$) (o denominador temporal deve ser estritamente positivo).
3. Se ($\sum_i w_i \leq 0$), substituir **todos** os pesos por (1).

4. Padronização e tensor de estrutura

4.1 Médias e variâncias ponderadas

Sejam ($v_{1,i} = \Delta t_i$) e ($v_{2,i} = \Delta p_i$) **antes** de padronizar. Com pesos (w_i):

$$\begin{aligned}\bar{v}_1 &= \frac{\sum_i w_i v_{1,i}}{\sum_i w_i}, & \bar{v}_2 &= \frac{\sum_i w_i v_{2,i}}{\sum_i w_i} \\ \sigma_1 &= \sqrt{\frac{\sum_i w_i (v_{1,i} - \bar{v}_1)^2}{\sum_i w_i}}, & \sigma_2 &= \sqrt{\frac{\sum_i w_i (v_{2,i} - \bar{v}_2)^2}{\sum_i w_i}} \\ \tilde{v}_{1,i} &= \frac{v_{1,i} - \bar{v}_1}{\max(\sigma_1, \varepsilon)}, & \tilde{v}_{2,i} &= \frac{v_{2,i} - \bar{v}_2}{\max(\sigma_2, \varepsilon)}\end{aligned}$$

Se `standardize=False`, usa-se ($\tilde{v}_{k,i} = v_{k,i}$) (sem centrar nem dividir).

4.2 Tensor

$$\mathbf{J} = \sum_i w_i \begin{bmatrix} \tilde{v}_{1,i}^2 & \tilde{v}_{1,i} \tilde{v}_{2,i} \\ \tilde{v}_{1,i} \tilde{v}_{2,i} & \tilde{v}_{2,i}^2 \end{bmatrix}$$

4.3 Autovalores e vector próprio principal

`numpy.linalg.eigh(J)` devolve autovalores em **ordem crescente**: $(\lambda_{\mathrm{small}} \leq \lambda_{\mathrm{large}})$. O código define:

$$\lambda_2 := \lambda_{\mathrm{small}}, \quad \lambda_1 := \lambda_{\mathrm{large}}$$

Seja $\mathbf{v} = (v_1, v_2)^{\mathrm{top}}$ o autovetor associado a (λ_1) (segunda coluna da matriz de autovectores devolvida). Então:

$$\mu = \mathrm{atan2}(v_2, v_1)$$

4.4 Anisotropia do tensor

Se $(\lambda_1 + \lambda_2 > 0)$:

$$A_{\mathrm{tensor}} = \frac{\lambda_1 - \lambda_2}{\lambda_1 + \lambda_2} \in [0, 1]$$

Caso contrário $(A_{\mathrm{tensor}} = \mu = \mathrm{NaN})$ (autovalores não informativos).

5. Métricas escalares: D, τ , A_{tensor} , μ , R

5.1 Drift

$$D = \frac{\sum_i w_i \Delta p_i}{\sum_i w_i |\Delta p_i|}$$

com convênção ($D=0$) se o denominador for (0). Intervalo típico $([-1, 1])$.

5.2 Tortuosidade

$$\tau = 1 - \frac{|\sum_i w_i \Delta p_i|}{\sum_i w_i |\Delta p_i|}$$

depois $(\tau \rightarrow \mathrm{clip}(\tau, 0, 1))$.

Relação: (D) e (τ) encerram informação complementar sobre o **primeiro momento** de (Δp) com sinal vs. magnitude.

5.3 Coerência angular R

Com $(\theta_i = \mathrm{atan2}(\Delta p_i, \Delta t_i^{\mathrm{star}}))$ nos valores **não padronizados**:

$$C = \frac{\sum_i w_i \cos \theta_i}{\sum_i w_i}, \quad S = \frac{\sum_i w_i \sin \theta_i}{\sum_i w_i}, \quad R = \sqrt{C^2 + S^2} \in [0, 1]$$

(R) é a **norma da média vectorial** no círculo; coincide com a **resultante circular** (Mardia & Jupp, 2000).

6. Intervalos de confiança (bootstrap)

6.1 Por transição (métricas por janela)

Se `bootstrap_ci=True` e $(n \geq 8) (N_MIN_BOOTSTRAP)$:

- Repetir ($B = 1000$) vezes ($N_BOOTSTRAP$):
 - amostra com reposição índices ($i \in \{1, \dots, n\}$);
 - recalculando (A_{tensor}) e (R) no subconjunto reamostrado (com a mesma lógica de padronização dentro da reamostra).
- IC 95%: percentis **2.5** e **97.5** das listas de valores finitos.
- **Semente fixa:** `numpy.random.default_rng(42)` para reprodutibilidade.

6.2 Agregação 2A (entre instrumentos)

Se `bootstrap_ci=True` e ≥ 2 instrumentos válidos:

- Reamostragem **ao nível dos instrumentos** (com reposição): cada bootstrap escolhe (J) instrumentos com reposição e recalcula a média ponderada (§7.1).
 - IC 95% para (A_{tensor}) e (R) a partir das (B) réplicas.
-

7. Agregações 2A e 2B

7.1 2A — Média ponderada por instrumento

Para escalares ($X \in \{D, \tau, A_{\mathrm{tensor}}, R\}$):

$$\bar{X} = \frac{\sum_j W_j X^{(j)}}{\sum_j W_j}$$

onde ($W_j = \mathrm{weight_sum}^{(j)}$) (soma dos pesos das transições na janela).

Para (**μ**) (média circular das orientações principais):

$$C_\mu = \frac{\sum_j W_j \cos \mu^{(j)}}{\sum_j W_j}, \quad S_\mu = \frac{\sum_j W_j \sin \mu^{(j)}}{\sum_j W_j}, \quad \bar{\mu} = \text{atan2}(S_\mu, C_\mu)$$

7.2 2B – Pool global

Concatenam-se verticalmente os DataFrames de transições de todas as partes e aplica-se **uma única** chamada a `compute_metrics_from_transitions` ao conjunto fundido.

8. Conflito direccional

Para uma janela (w), com instrumentos (j) com $(\mu^{(j)})$ e pesos ($W_j = \text{weight_sum}^{(j)}$) finitos:

$$C_{\text{inst}} = \frac{\sum_j W_j \cos \mu^{(j)}}{\sum_j W_j}, \quad S_{\text{inst}} = \frac{\sum_j W_j \sin \mu^{(j)}}{\sum_j W_j}$$

$$R_{\text{inst}} = \sqrt{C_{\text{inst}}^2 + S_{\text{inst}}^2}, \quad \text{Conflito}(w) = 1 - R_{\text{inst}}$$

Interpretação: $(\text{Conflito} \approx 0)$ — orientações (μ) alinhadas; (≈ 1) — (μ) em direcções opostas ou muito dispersas.

9. Janelamento temporal

9.1 Modos

Modo	Parâmetros	Conteúdo da janela
total	—	Todas as transições da parte
measures	<code>window_size, step</code> (inteiros)	$(\text{meas} \in [m_0, m_0 + \text{size}))$
seconds	<code>window_size, step</code> (float)	$(t \in [t_{\text{cur}}, t_{\text{cur}} + \text{size}))$
events	<code>window_size, step</code> (inteiros)	Fatias <code>iloc[i_0 : i_0 + size)</code>

9.2 Fallback medidas → segundos

Se **todos** os (meas) na parte forem (0), o modo `measures` deixa de ser aplicável e o código continua com a lógica de **seconds** sobre a coluna `t`.

9.3 Ordenação de etiquetas (`window_sort_key`)

Extrai-se um número para ordenação cronológica a partir do prefixo da etiqueta (`m...`, `t...`, `e...`, ou `total` → 0).

10. Visualizações (geometria)

10.1 Elipses do tensor

Largura e altura de ellipse (exibição):

$$\text{width} = 2\sqrt{\lambda_1} s, \quad \text{height} = 2\sqrt{\lambda_2} s$$

com escala (s) arbitrária (`scale` no código). Ângulo de rotação: (μ) em graus.

Fallback quando (λ_1, λ_2) não estão disponíveis (alguns agregados):

$$\lambda'_1 = \max\left(\frac{1+A}{2}, 10^{-6}\right), \quad \lambda'_2 = \max\left(\frac{1-A}{2}, 10^{-6}\right)$$

(traço normalizado; preserva a razão $(\lambda_1 - \lambda_2)/(\lambda_1 + \lambda_2)$.)

10.2 Rose diagram

Histograma polar de (θ_i) em (n_{bins}) bins em $([-\pi, \pi])$. Modo por janela: repete-se para cada subconjunto de transições.

10.3 Flow map (quiver)

Numa grelha (janela $\times$ instrumento): ($U \propto A \cos \mu$), ($V \propto A \sin \mu$); cor mapeada a partir de (D) via $((D+1)/2)$ quando finito.

11. Algoritmos (pseudocódigo)

11.1 `compute_metrics_from_transitions(df, time_axis, weight_mode, standardize, bootstrap_ci)`

ENTRADA: `df`, `time_axis` $\in \{"ql", "sec"\}$, `weight_mode` $\in \{"dur", "min"\}$

SAÍDA: Metrics

1. Filtrar `dp`, `dt_col` finitos; manter apenas `dt_col > 0`.
2. `dp` $\leftarrow$ vetor; `dt` $\leftarrow$ vetor da coluna `dt_ql` ou `dt_sec`; `w` $\leftarrow$ `w_dur` ou `w_min`.
3. `w` $\leftarrow$ $\max(w, 0)$; se $\sum(w) \leq 0$ então `w` $\leftarrow$ ones.
4. `D` $\leftarrow$ $\sum(w*dp) / \sum(w*|dp|)$ (ou 0 se denominador 0)
5. `tau` $\leftarrow$ $\text{clip}(1 - |\sum(w*dp)| / \sum(w*|dp|), 0, 1)$
6. `(A, mu, R, lambda1, lambda2)` $\leftarrow$ `_compute_tensor_and_R_internal(dt, dp, w, standardize)`
// `theta` e `R` usam `dp` e `dt` ORIGINAIS dentro desta função
7. Se `bootstrap_ci` e `n` ≥ 8 :
 repetir 1000 vezes:
 `idx` $\leftarrow$ amostra aleatória 1..n com reposição
 calcular `A`, `R` no subconjunto `idx`
 `IC_A` $\leftarrow$ percentis 2.5, 97.5 dos `A`; idem `R`
8. Retornar `Metrics(...)`

11.2 `_compute_tensor_and_R_internal(dt, dp, w, standardize)`

1. Se $\sum(w) \leq 0 \rightarrow$ retornar `(NaN, ..., NaN)`.
2. Se `standardize`: substituir `v1=dt`, `v2=dp` por versões centradas e divididas por σ (cor
3. Montar `J`; `eigh` $\rightarrow \lambda_2 \leq \lambda_1$; $\mu \leftarrow \text{atan2}$ do autovector de λ_1 .
4. `A` $\leftarrow (\lambda_1 - \lambda_2) / (\lambda_1 + \lambda_2)$ se $\lambda_1 + \lambda_2 > 0$ senão `NaN`.
5. `theta_i` $\leftarrow \text{atan2}(dp_i, dt_i)$ // `dp` e `dt` ORIGINAIS (não padronizados)
6. `C` $\leftarrow \sum(w*\cos(\theta)) / \sum(w)$; `S` $\leftarrow$ idem `sin`; `R` $\leftarrow \sqrt{C^2 + S^2}$
7. Retornar `(A, mu, R, lambda1, lambda2)`

12. Tutorial

12.1 Pré-requisitos

- Python 3.10+ recomendado (CI usa 3.10).
- Dependências: ver `requirements.txt` (`streamlit`, `numpy`, `pandas`, `matplotlib`, `music21`, `pytest`).
- Partitura de teste: `tests/fixtures/minimal_score.xml` ou qualquer `.xml` / `.musicxml` / `.mxl`.

12.2 Interface Streamlit (recomendado para exploração)

Na pasta do projecto:

```
pip install -r requirements.txt
streamlit run Anisotropia.py
```

1. **Upload** do MusicXML.
2. Escolher **representante de acorde** (centroid / top / bottom).
3. Escolher **peso** (dur vs min) e **modo de janela** (compassos, segundos, transições, excerto total).
4. Activar **modo científico** para padronização + IC 95% quando aplicável.
5. Ler a tabela de resultados, exportar **CSV** e **relatório** Markdown (generate_report em inglês).

12.3 Uso programático mínimo

```
from pathlib import Path
from anisotropia.parsing import parse_musicxml, transitions_from_events
from anisotropia.metrics import compute_metrics_from_transitions
from anisotropia.windowing import window_slices_for_part

xml_bytes = Path("ficheiro.xml").read_bytes()
events_by_part, has_seconds = parse_musicxml(xml_bytes, "ficheiro.xml", chord_rep="cent")

# Parte com mais transições como referência para janelas
def transitions_map(ebp):
    return {k: transitions_from_events(v) for k, v in ebp.items()}

trans = transitions_map(events_by_part)
ref = max(trans.keys(), key=lambda k: len(trans[k]))
windows = window_slices_for_part(trans[ref], "measures", window_size=4.0, step=2.0)

time_axis = "sec" if has_seconds else "ql"
for label, _ in windows[:3]:
    df_w = trans[ref] # na app: alinhar cortes por medida/tempo/evento como em Anisotrópia
    m = compute_metrics_from_transitions(
        df_w, time_axis=time_axis, weight_mode="dur",
        standardize=True, bootstrap_ci=len(df_w) >= 8,
    )
    print(label, m.A_tensor, m.R, m.n)
```

Para relatórios Markdown use **generate_report** (não build_report):

```
from anisotropia.report import generate_report
import pandas as pd

text = generate_report(
    filename="exemplo.xml",
    df_results=pd.DataFrame(...), # colunas como na app
```

```

params={"chord_rep": "centroid", "weight_mode": "dur", ...},
n_parts=len(events_by_part),
n_windows=5,
total_transitions=len(trans[ref]),
)
Path("relatorio.md").write_text(text, encoding="utf-8")

```

12.4 Exemplo numérico (mão)

Três transições, pesos ($w_i=1$), ($\Delta p \in \{2,-1,1\}$):

$$\sum w_i \Delta p_i = 2, \quad \sum w_i |\Delta p_i| = 4$$

$$D = \frac{2}{4} = 0.5, \quad \tau = 1 - \frac{|2|}{4} = 0.5$$

Para (A_{tensor}) e (μ) é necessário o par ($(\Delta t_i, \Delta p_i)$) completo e a padronização; o exemplo ilustra apenas **(D)** e **(τ)**.

12.5 Interpretação rápida

Métrica	Pergunta que responde
(D)	A melodia tende a subir ou descer em média?
(τ)	A linha zigzagueia (cancelamento de subidas/descidas)?
(A_{tensor})	Existe um eixo dominante no plano ($(\Delta t, \Delta p)$) após padronizar?
(μ)	Qual a direcção desse eixo?
(R)	Os ângulos (θ_i) estão alinhados (resultante circular forte)?

12.6 Problemas frequentes

Sintoma	Causa provável
Janelas em segundos vazias	Ficheiro sem tempos absolutos fiáveis; usar <code>q1</code> ou compassos.
(n) baixo / IC vazio	Poucas transições na janela; aumentar janela ou usar excerto total.
meas sempre 0	Alguns exports MusicXML; janelas por compassos degradam para segundos.

13. Apêndices

Apêndice A — Constantes (`metrics.py`)

Nome	Valor
N_MIN_BOOTSTRAP	8
N_MIN_STABLE	15 (recomendação na UI)
N_BOOTSTRAP	1000
EPSILON	(10^{-9})

Apêndice B — Limite de ficheiro

`MAX_FILE_SIZE_BYTES = 50 \times 1024^2` — rejeição explícita acima deste tamanho.

Apêndice C — Correspondência expressão ↔ código

Conceito	Local típico
$(D), (\tau)$	<code>metrics.py</code> após filtragem
$(\mathbf{J}), (\lambda), (\mu), (A)$	<code>_compute_tensor_and_R_internal</code>
$(\theta), (R)$	mesma função, usa <code>np.arctan2(dp, dt)</code>
2A	<code>_compute_weighted_aggregate, aggregate_2A</code>
2B	<code>aggregate_2B</code>
Conflito	<code>compute_directional_conflict</code>

Apêndice D — Documentação complementar

- `MANUAL_METRICAS.md` — resumo das métricas para a barra lateral da app Streamlit (não substitui este manual).
-

14. Bibliografia

1. Bigün, J., & Granlund, G. H. (1987). Optimal orientation detection of linear symmetry. *ICCV*.
 2. Mardia, K. V., & Jupp, P. E. (2000). *Directional Statistics*. Wiley.
 3. Efron, B., & Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
 4. Woodcock, N. H. (1977). Eigenvalue method for fabric shape. *GSA Bulletin*.
 5. Basser, P. J., Mattiello, J., & LeBihan, D. (1994). MR diffusion tensor. *J. Magn. Reson.*
 6. Cuthbert, M. S., & Ariza, C. (2010). music21. *ISMIR*.
 7. Weickert, J. (1998). *Anisotropic Diffusion in Image Processing*. Teubner.
-

Fim do manual. Para alterações de comportamento, consultar sempre os testes em `tests/` e as funções referidas nas secções 11–13.

